

student analysis

March 21, 2026

1 Student Performance Analysis

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: # Set professional global plotting theme
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update({'figure.titlesize': 16, 'axes.titlesize': 14, 'axes.
↳labelsizesize': 12})
```

1.1 Data Loading and Cleaning

```
[3]: df=pd.read_csv("student_score.csv")
print(df.head())
```

	Unnamed: 0	Gender	EthnicGroup	ParentEduc	LunchType	TestPrep	\
0	0	female	NaN	bachelor's degree	standard	none	
1	1	female	group C	some college	standard	NaN	
2	2	female	group B	master's degree	standard	none	
3	3	male	group A	associate's degree	free/reduced	none	
4	4	male	group C	some college	standard	none	

	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	TransportMeans	\
0	married	regularly	yes	3.0	school_bus	
1	married	sometimes	yes	0.0	NaN	
2	single	sometimes	yes	4.0	school_bus	
3	married	never	no	1.0	NaN	
4	married	sometimes	yes	0.0	school_bus	

	WklyStudyHours	MathScore	ReadingScore	WritingScore
0	< 5	71	71	74
1	5 - 10	69	90	88
2	< 5	87	93	91
3	5 - 10	45	56	42
4	5 - 10	76	78	75

```
[4]: df.describe()
```

```
[4]:
```

	Unnamed: 0	NrSiblings	MathScore	ReadingScore	WritingScore
count	30641.000000	29069.000000	30641.000000	30641.000000	30641.000000
mean	499.556607	2.145894	66.558402	69.377533	68.418622
std	288.747894	1.458242	15.361616	14.758952	15.443525
min	0.000000	0.000000	0.000000	10.000000	4.000000
25%	249.000000	1.000000	56.000000	59.000000	58.000000
50%	500.000000	2.000000	67.000000	70.000000	69.000000
75%	750.000000	3.000000	78.000000	80.000000	79.000000
max	999.000000	7.000000	100.000000	100.000000	100.000000

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30641 entries, 0 to 30640
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            30641 non-null  int64
1   Gender                30641 non-null  object
2   EthnicGroup           28801 non-null  object
3   ParentEduc            28796 non-null  object
4   LunchType             30641 non-null  object
5   TestPrep              28811 non-null  object
6   ParentMaritalStatus   29451 non-null  object
7   PracticeSport         30010 non-null  object
8   IsFirstChild          29737 non-null  object
9   NrSiblings            29069 non-null  float64
10  TransportMeans        27507 non-null  object
11  WklyStudyHours        29686 non-null  object
12  MathScore             30641 non-null  int64
13  ReadingScore          30641 non-null  int64
14  WritingScore          30641 non-null  int64
dtypes: float64(1), int64(4), object(10)
memory usage: 3.5+ MB
```

```
[6]: df.isnull().sum()
```

```
# Categorical: Fill with Most Frequent (Mode)
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# Numerical: Fill with Median (more robust against outliers than Mean)
df["NrSiblings"] = df["NrSiblings"].fillna(df["NrSiblings"].median())

# Drop irrelevant index columns immediately
df = df.drop(columns=["Unnamed: 0"], errors="ignore")
```

```
[7]: print(df.head())
```

	Gender	EthnicGroup	ParentEduc	LunchType	TestPrep	\
0	female	group C	bachelor's degree	standard	none	
1	female	group C	some college	standard	none	
2	female	group B	master's degree	standard	none	
3	male	group A	associate's degree	free/reduced	none	
4	male	group C	some college	standard	none	

	ParentMaritalStatus	PracticeSport	IsFirstChild	NrSiblings	TransportMeans	\
0	married	regularly	yes	3.0	school_bus	
1	married	sometimes	yes	0.0	school_bus	
2	single	sometimes	yes	4.0	school_bus	
3	married	never	no	1.0	school_bus	
4	married	sometimes	yes	0.0	school_bus	

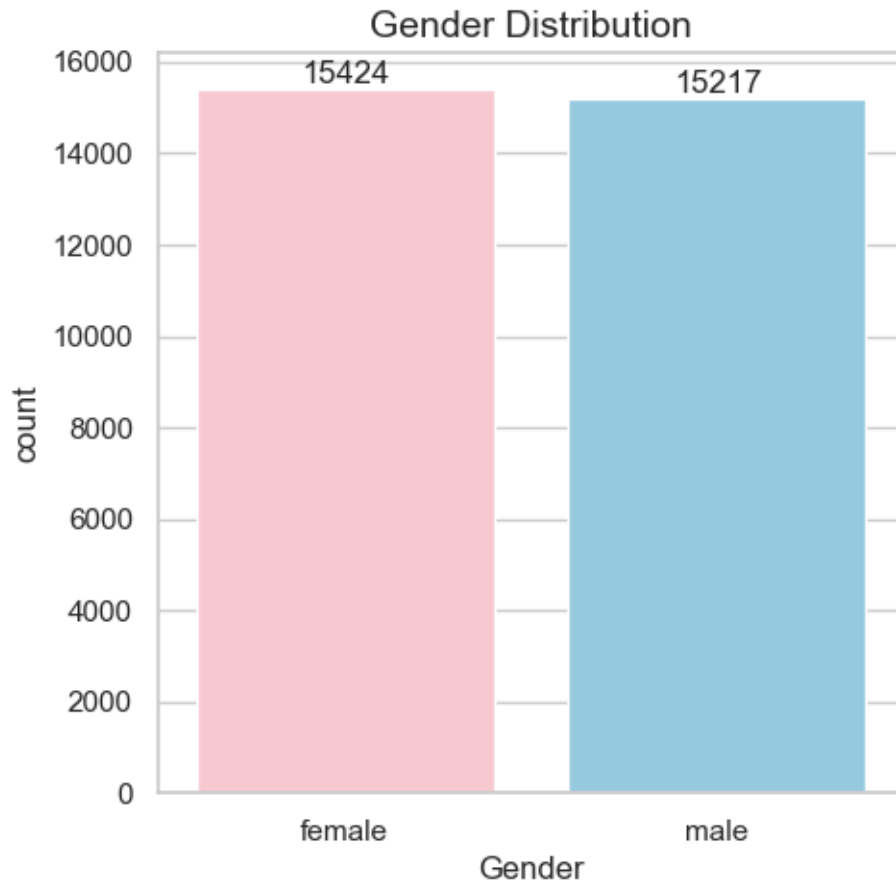
	WklyStudyHours	MathScore	ReadingScore	WritingScore
0	< 5	71	71	74
1	5 - 10	69	90	88
2	< 5	87	93	91
3	5 - 10	45	56	42
4	5 - 10	76	78	75

1.2 Gender Distribution

```
[8]: #gender distribution
plt.figure(figsize=(5,5))
plt.title("Gender Distribution")
ax = sns.countplot(
    data=df,
    x="Gender",
    hue="Gender",
    palette={"male": "skyblue", "female": "pink"},
    legend=False
)

ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])

plt.show()
```



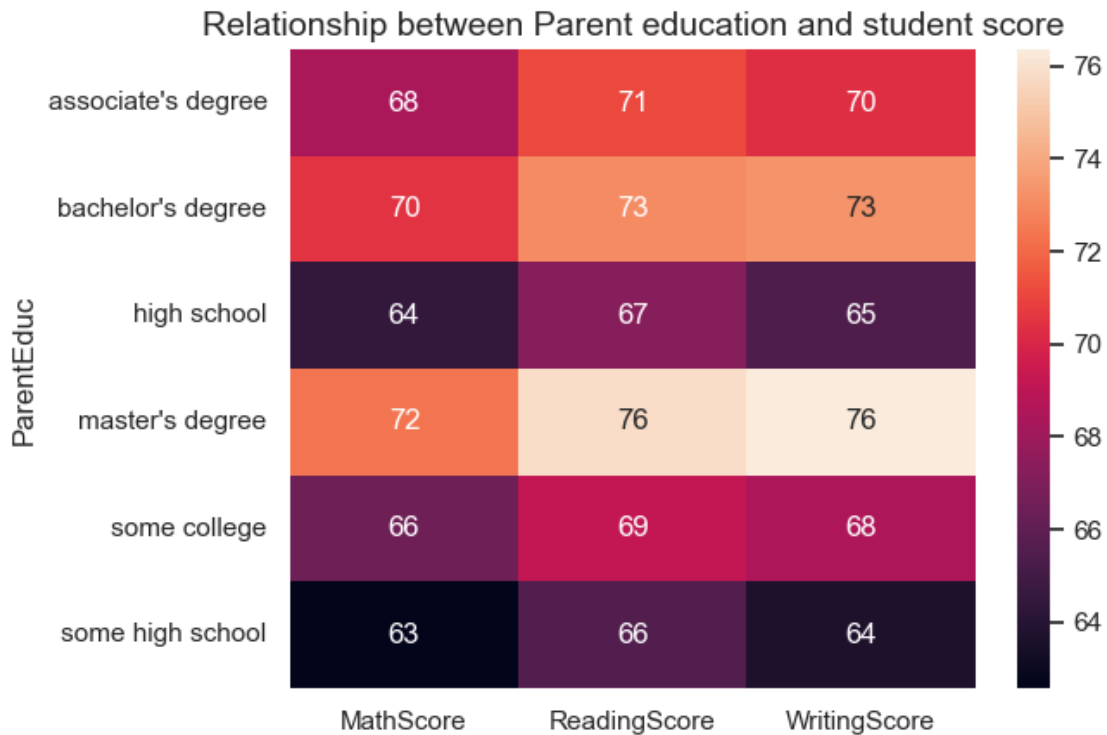
2 From the above chart, the number of females in the data is slightly higher than the number of males.

2.1 Parent Education and Student Performance

```
[9]: gp0=df.groupby("ParentEduc").agg({"MathScore":'mean',"ReadingScore":
    ↳'mean',"WritingScore":'mean'})
print(gp0)
```

	MathScore	ReadingScore	WritingScore
ParentEduc			
associate's degree	68.365586	71.124324	70.299099
bachelor's degree	70.466627	73.062020	73.331069
high school	64.435731	67.213997	65.421136
master's degree	72.336134	75.832921	76.356896
some college	66.445978	69.189667	68.456711
some high school	62.584013	65.510785	63.632409

```
[10]: sns.heatmap(gp0,annot=True)
plt.title("Relationship between Parent education and student score")
plt.show()
```



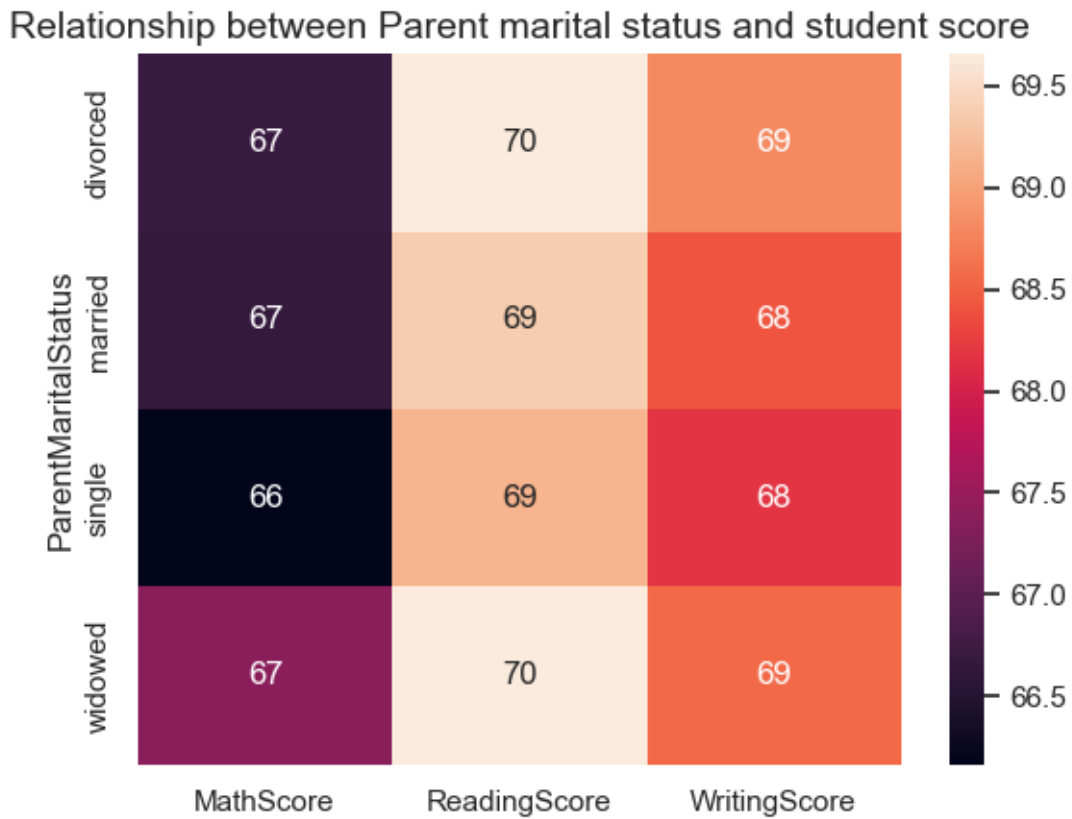
3 From the above chart, we can see that parents' education has an impact on student scores.

#student whose parents have higher education levels usually score better, especially in reading and writing

```
[11]: gp1=df.groupby("ParentMaritalStatus").agg({"MathScore": 'mean', "ReadingScore":
→ 'mean', "WritingScore": 'mean'})
print(gp1)
```

ParentMaritalStatus	MathScore	ReadingScore	WritingScore
divorced	66.691197	69.655011	68.799146
married	66.650161	69.379561	68.406177
single	66.165704	69.157250	68.174440
widowed	67.368866	69.651438	68.563452

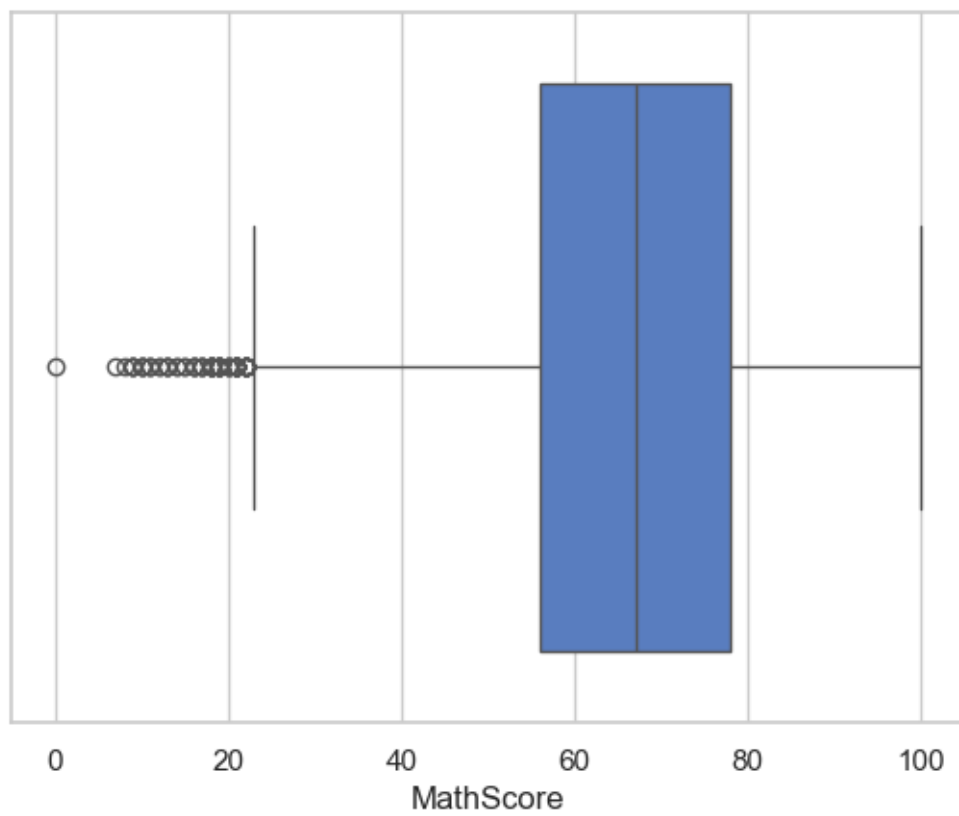
```
[12]: sns.heatmap(gp1,annot=True)
plt.title("Relationship between Parent marital status and student score")
plt.show()
```

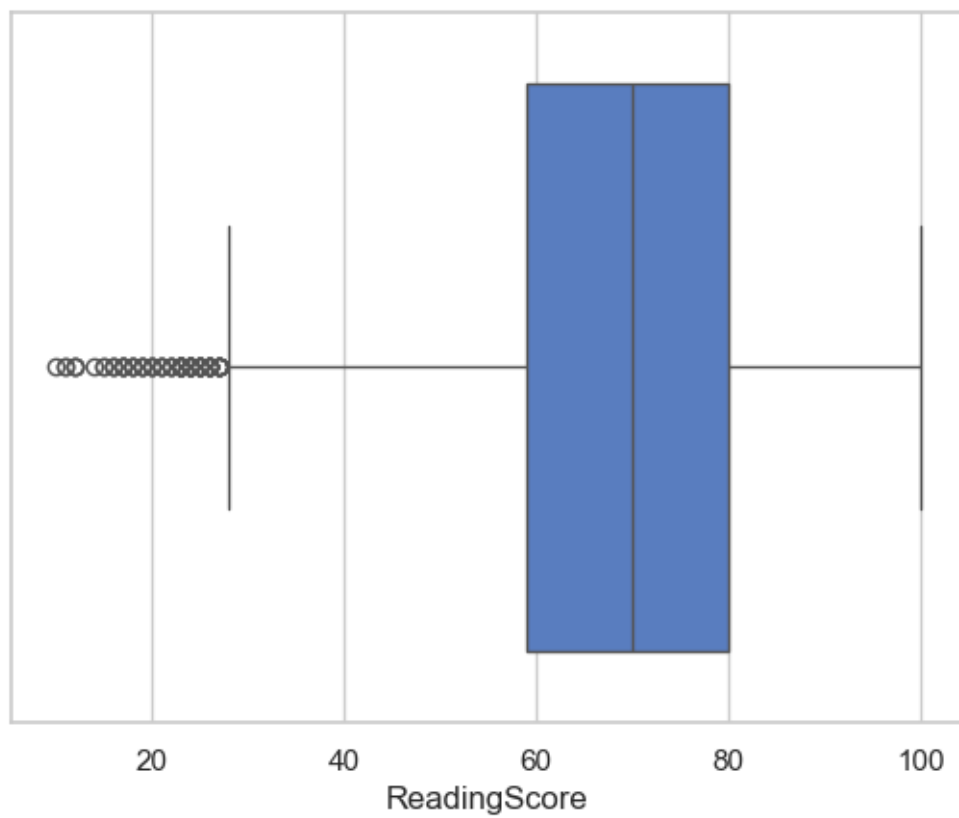


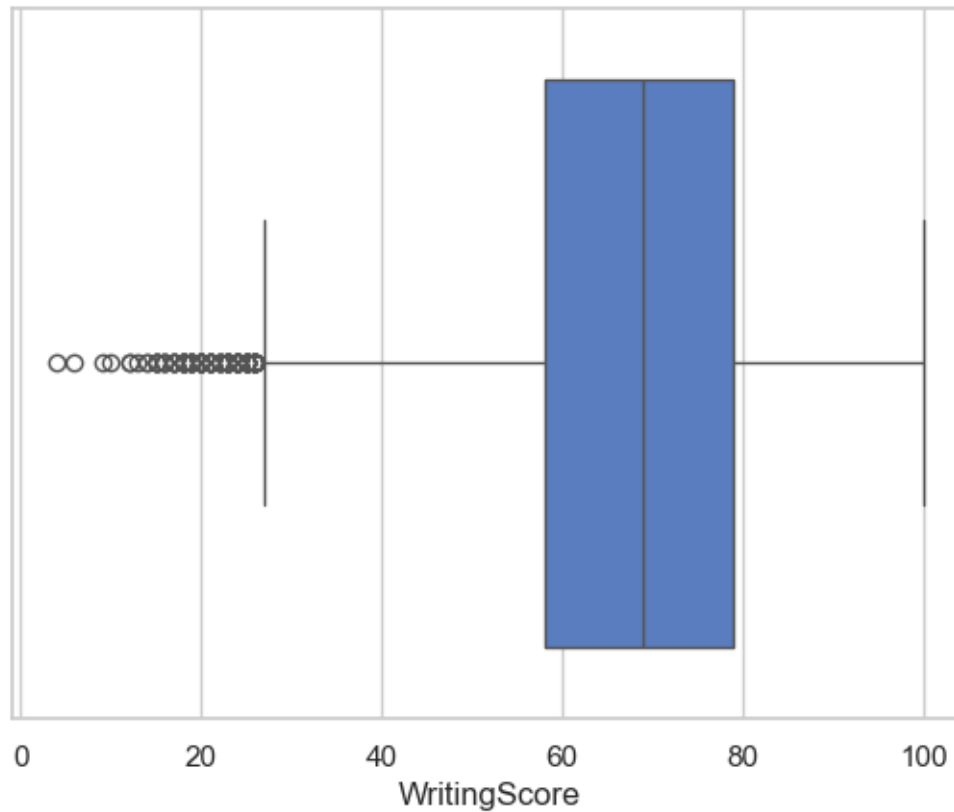
4 Parent marital status has little or no impact on student scores.

4.1 Subject-wise Score Distribution

```
[13]: sns.boxplot(data=df,x="MathScore")
plt.show()
sns.boxplot(data=df,x="ReadingScore")
plt.show()
sns.boxplot(data=df,x="WritingScore")
plt.show()
```







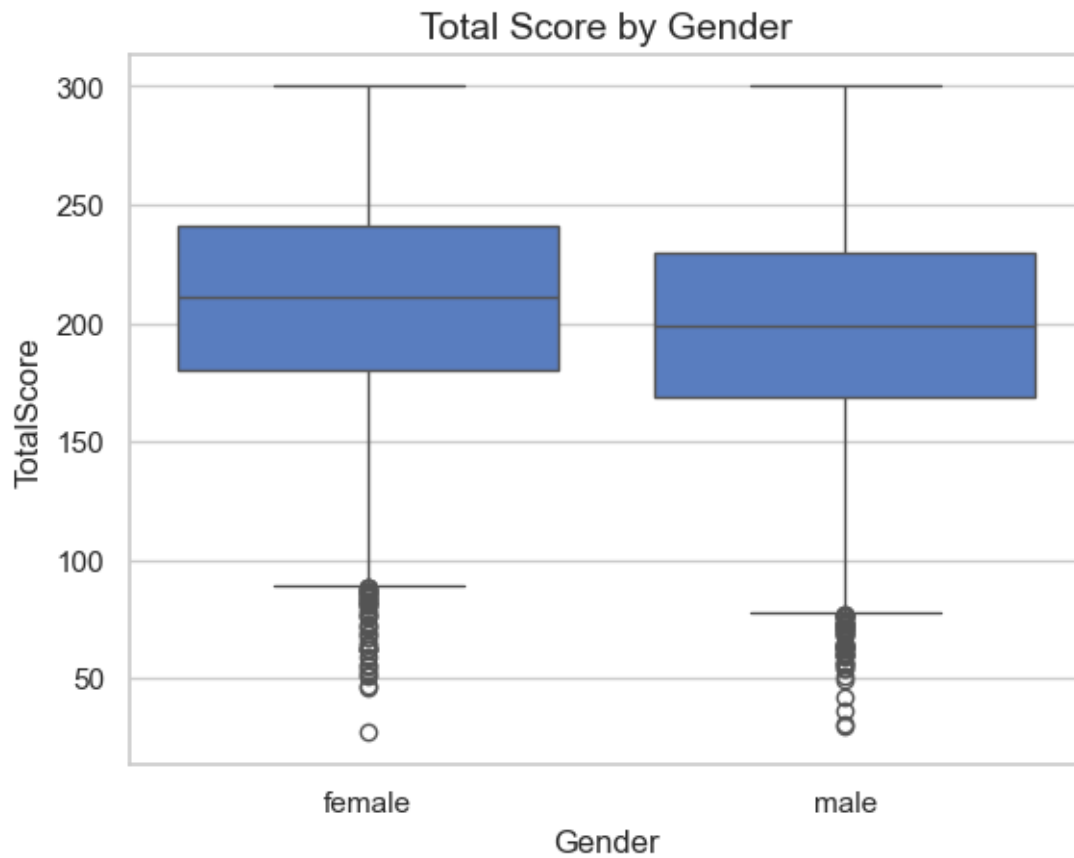
5 Math appears to be relatively more difficult than reading and writing.

```
[14]: #total score section
df["TotalScore"] = df["MathScore"] + df["ReadingScore"] + df["WritingScore"]
```

```
[15]: # Average score
df["AvgScore"] = df["TotalScore"] / 3

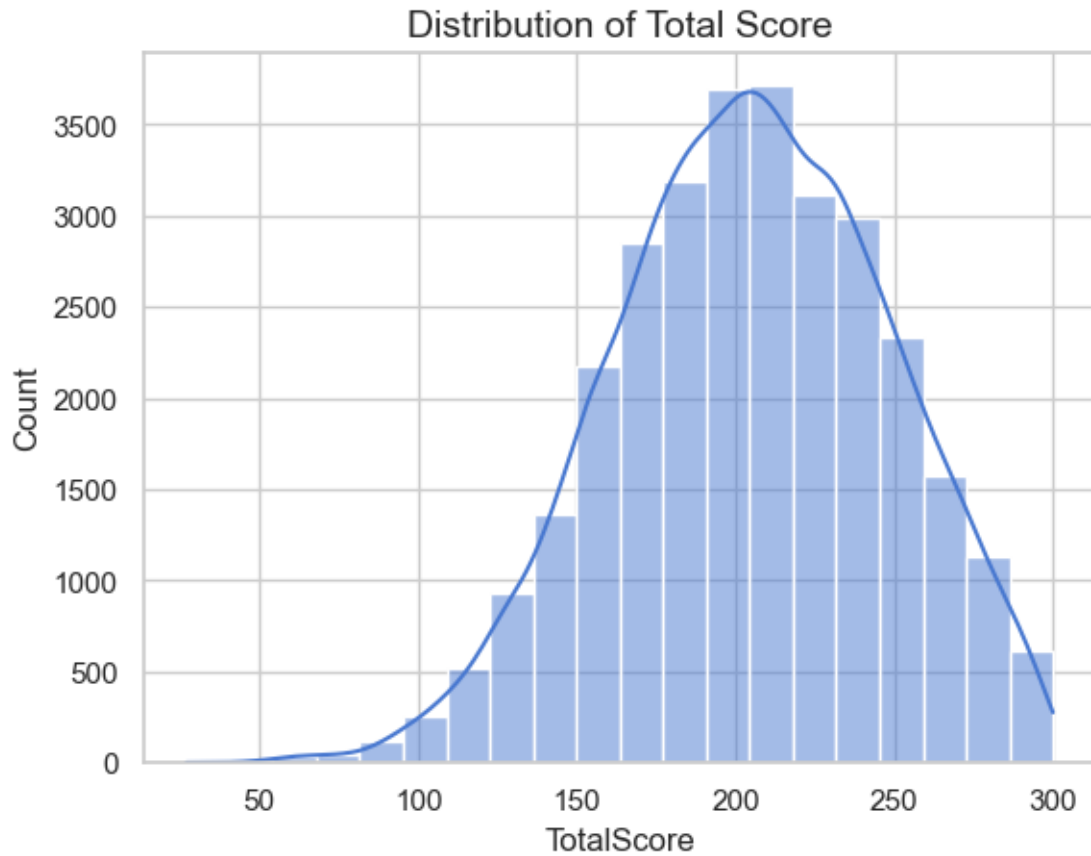
# Performance category
df["Performance"] = pd.cut(
    df["AvgScore"],
    bins=[0, 50, 70, 85, 100],
    labels=["Poor", "Average", "Good", "Excellent"]
)
```

```
[16]: sns.boxplot(data=df, x="Gender", y="TotalScore")
plt.title("Total Score by Gender")
plt.show()
```



#The total scores of male and female students are quite close, but females have a slightly higher median score. #This means both genders perform almost equally in academics.

```
[17]: sns.histplot(df["TotalScore"], bins=20, kde=True)
plt.title("Distribution of Total Score")
plt.show()
```



#Most students score in the middle range, with fewer students scoring extremely high or low.

#Total score gives an overall idea of student performance instead of looking at each subject separately

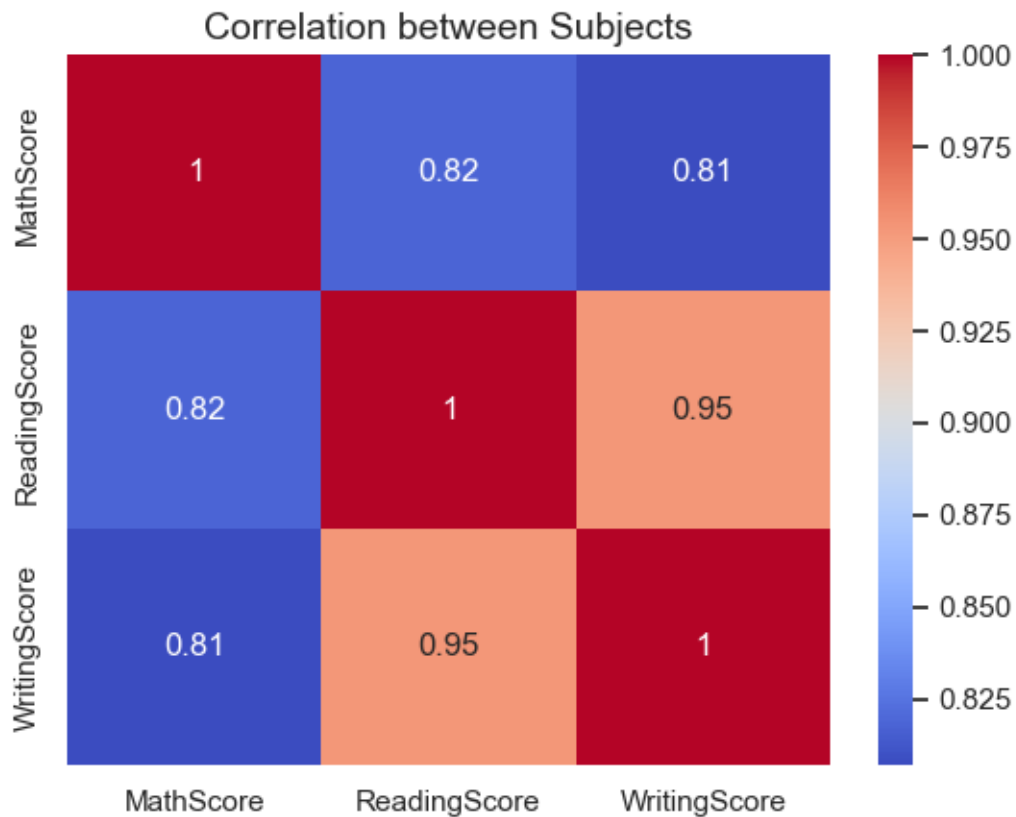
5.1 Standardized Score (Z-Score)

```
[18]: # Z-score for Total Score
df["ZScore"] = (df["TotalScore"] - df["TotalScore"].mean()) / df["TotalScore"].
    →std()
df[["TotalScore", "ZScore"]].head()
```

```
[18]:   TotalScore   ZScore
0         216  0.268557
1         247  0.983453
2         271  1.536920
3         143 -1.414906
4         229  0.568352
```

5.2 Relationship Between Subjects

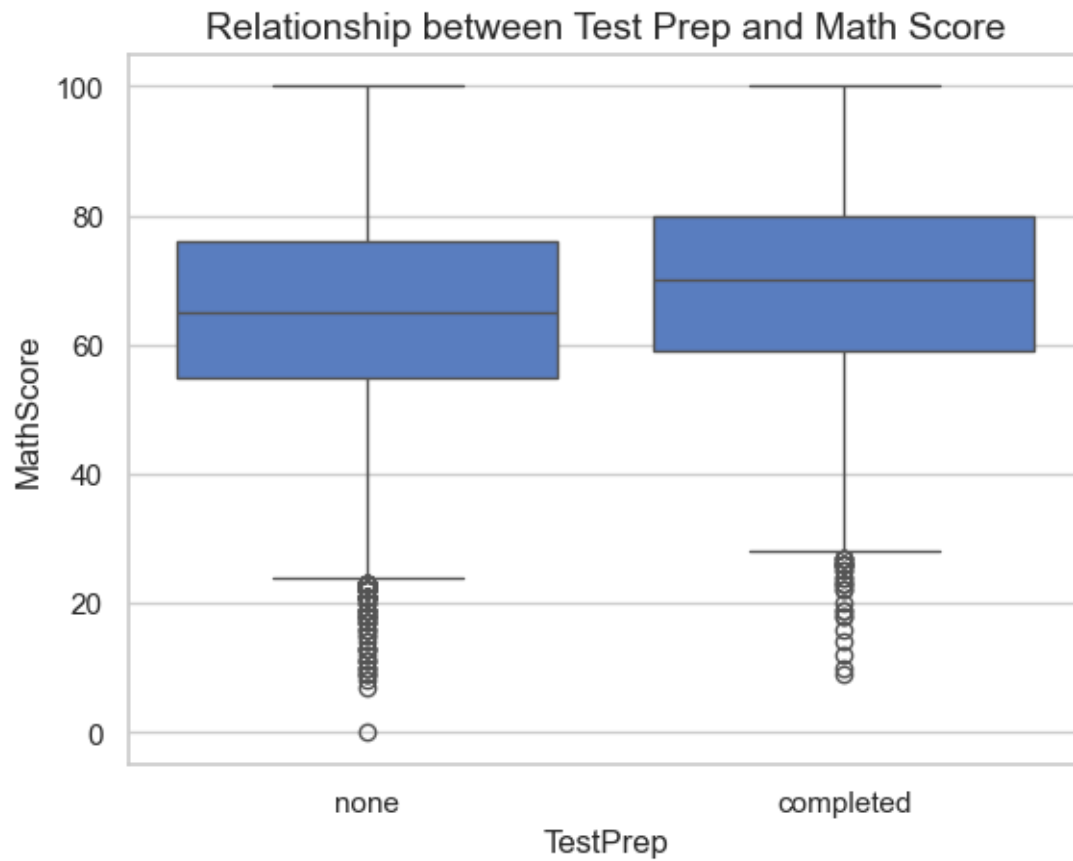
```
[19]: #Correlation Section
corr = df[["MathScore", "ReadingScore", "WritingScore"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation between Subjects")
plt.show()
```



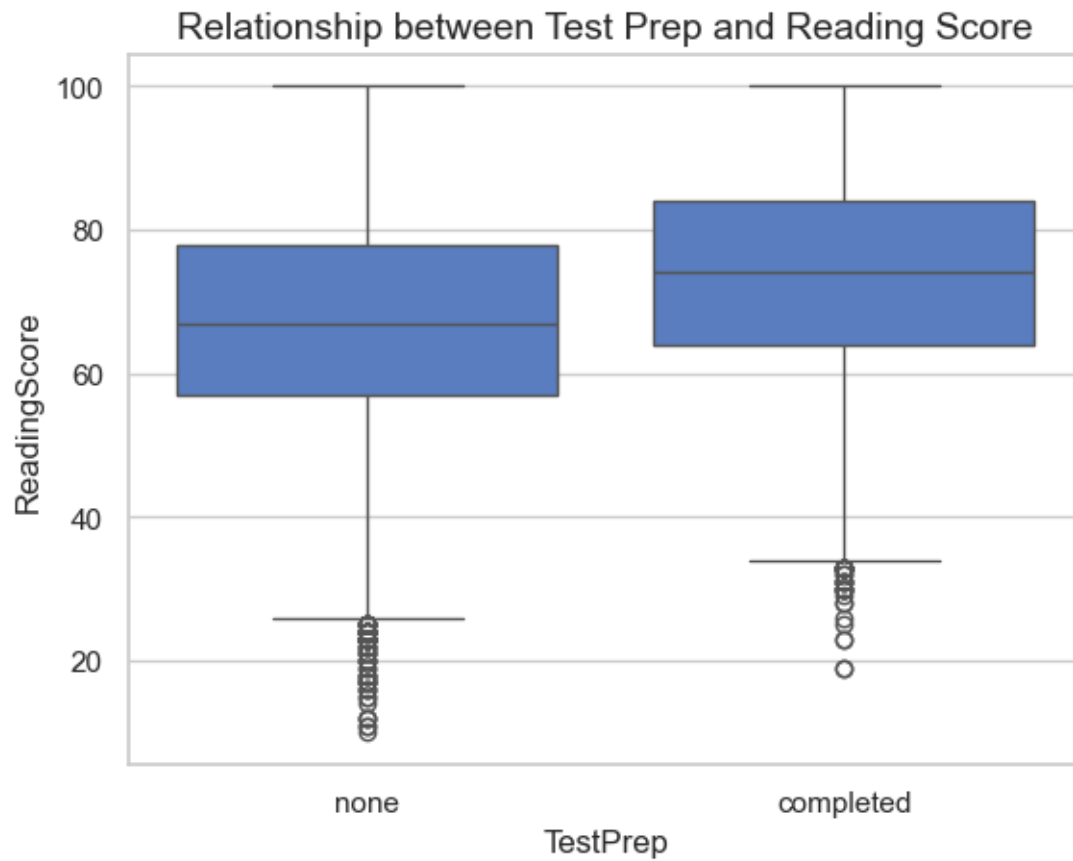
6 Reading and writing scores are closely related, while math is slightly different.

6.1 Effect of Test Preparation

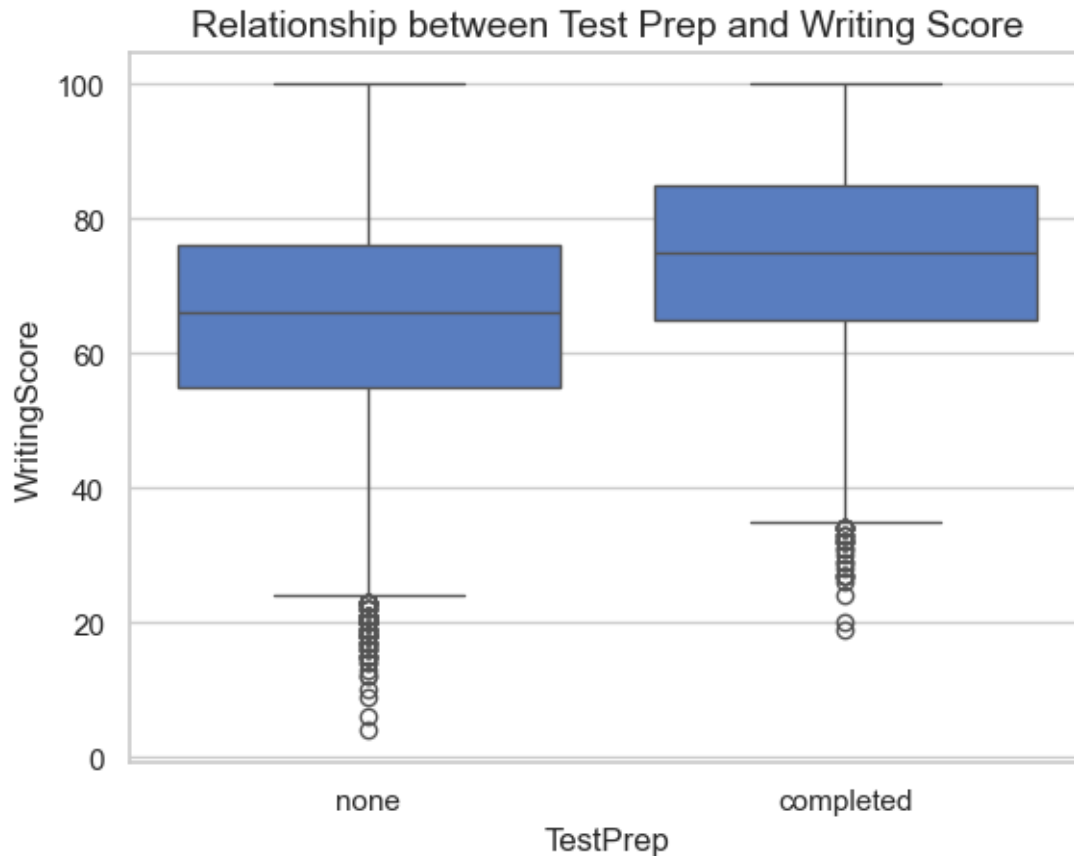
```
[20]: sns.boxplot(data=df, x="TestPrep", y="MathScore")
plt.title("Relationship between Test Prep and Math Score")
plt.show()
```



```
[21]: sns.boxplot(data=df,x="TestPrep",y="ReadingScore")  
plt.title("Relationship between Test Prep and Reading Score")  
plt.show()
```



```
[22]: sns.boxplot(data=df,x="TestPrep",y="WritingScore")  
plt.title("Relationship between Test Prep and Writing Score")  
plt.show()
```



```
[23]: prep_scores = df.groupby("TestPrep")["TotalScore"].mean()
print(prepare_scores)

improvement = (
    (prep_scores["completed"] - prep_scores["none"]) /
    prep_scores["none"]
) * 100

print(f"Score improvement due to test prep: {improvement:.2f}%")
```

```
TestPrep
completed    217.982923
none         197.795997
Name: TotalScore, dtype: float64
Score improvement due to test prep: 10.21%
```

7 T - TEST

```
[25]: from scipy.stats import ttest_ind

# Scientific proof: Does test prep actually work?
group_prep = df[df["TestPrep"] == "completed"]["TotalScore"]
group_no_prep = df[df["TestPrep"] == "none"]["TotalScore"]

t_stat, p_val = ttest_ind(group_prep, group_no_prep)
print(f"Statistical Significance (p-value): {p_val:.4f}")

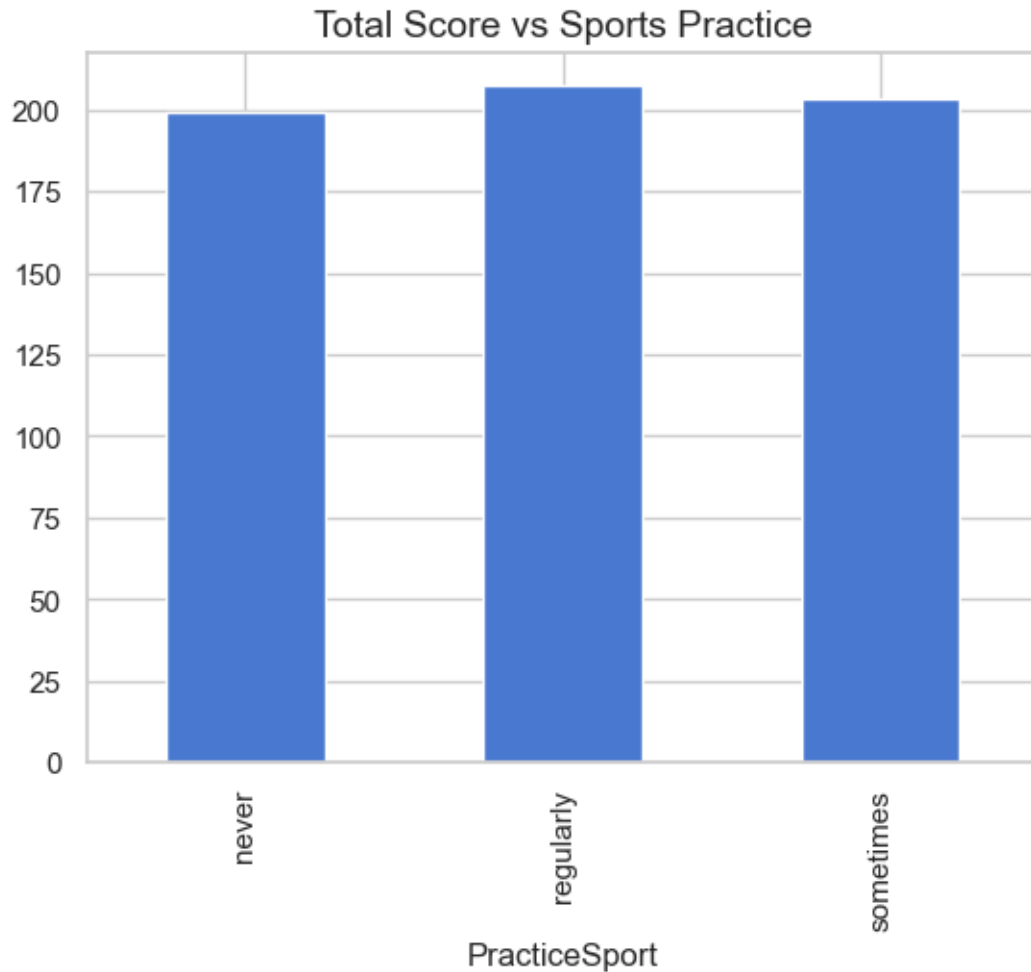
if p_val < 0.05:
    print("Insight: The impact of test preparation is statistically significant.
    ↪")
```

Statistical Significance (p-value): 0.0000

Insight: The impact of test preparation is statistically significant.

7.1 Sports Activity and Performance

```
[26]: #sports vs performance
gp2 = df.groupby("PracticeSport")["TotalScore"].mean()
gp2.plot(kind="bar")
plt.title("Total Score vs Sports Practice")
plt.show()
```

#Students who practice sports regularly seem to perform slightly better.

```
[27]: print(df['EthnicGroup'].unique())
```

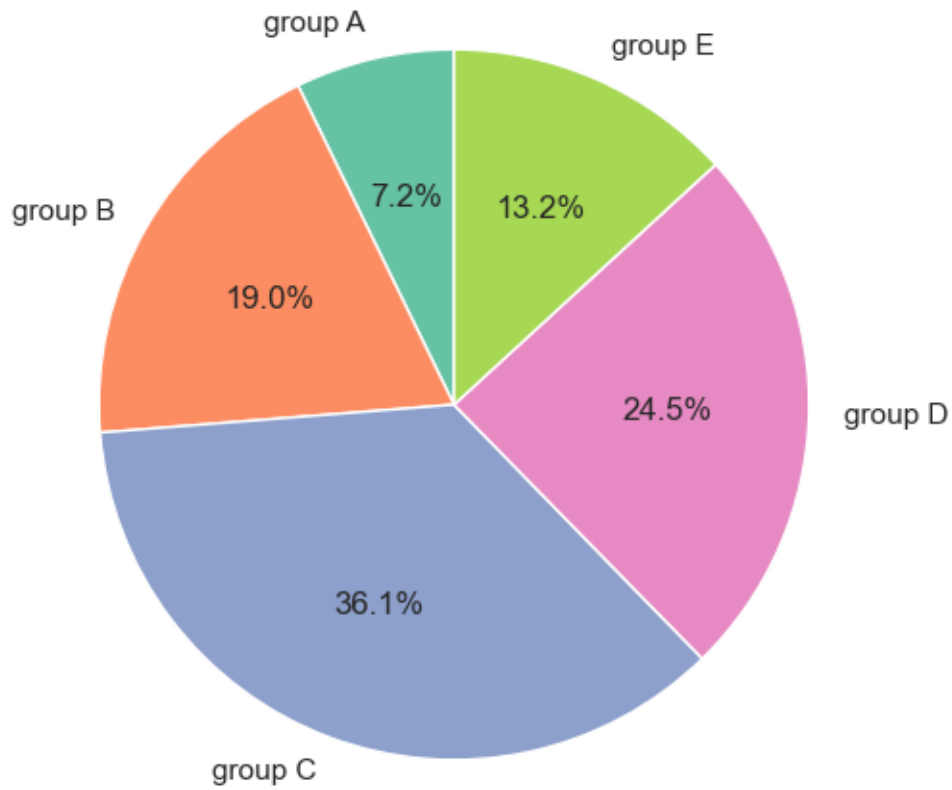
```
['group C' 'group B' 'group A' 'group D' 'group E']
```

```
[28]: #distribution of ethnic group
```

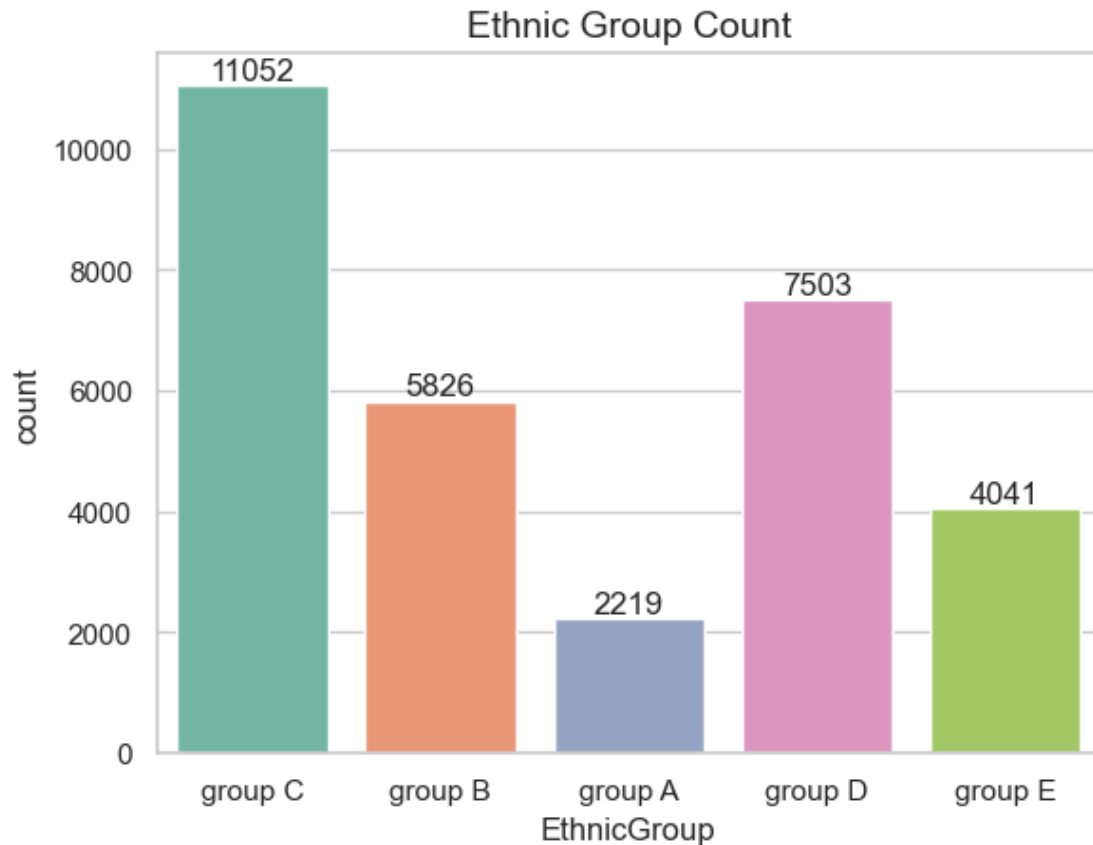
```
ethnic_counts = df["EthnicGroup"].value_counts().sort_index()

colors = ["#66c2a5", "#fc8d62", "#8da0cb", "#e78ac3", "#a6d854"]
plt.figure(figsize=(6, 6))
plt.pie(ethnic_counts, labels=ethnic_counts.index, autopct='%1.1f%%',
    ↪ colors=colors, startangle=90)
plt.title("Distribution of Ethnic Groups", fontsize=14, fontweight='bold')
plt.show()
```

Distribution of Ethnic Groups



```
[29]: ax=sns.countplot(data=df,
    ↳x="EthnicGroup",hue="EthnicGroup",palette=colors,legend=False)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Ethnic Group Count")
plt.show()
```



```
[30]: df.sort_values("TotalScore", ascending=False)[
        ↳
        ↳ ["Gender", "EthnicGroup", "MathScore", "ReadingScore", "WritingScore", "TotalScore"]
    ].head(5)
```

```
[30]:
```

	Gender	EthnicGroup	MathScore	ReadingScore	WritingScore	TotalScore
2484	female	group A	100	100	100	300
20396	female	group A	100	100	100	300
2325	male	group E	100	100	100	300
8883	male	group D	100	100	100	300
20498	female	group D	100	100	100	300

```
[31]: df.sort_values("TotalScore")[
        ↳
        ↳ ["Gender", "EthnicGroup", "MathScore", "ReadingScore", "WritingScore", "TotalScore"]
    ].head(5)
```

```
[31]:
```

	Gender	EthnicGroup	MathScore	ReadingScore	WritingScore	TotalScore
55	female	group C	0	17	10	27
14494	male	group D	7	11	12	30

18385	male	group B	10	12	9	31
4719	male	group B	19	11	6	36
30389	male	group B	24	14	4	42

#Top and bottom scoring students to understand the performance range.

7.2 MACHINE LEARNING MODEL

```
[32]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# Goal: Predict Math Score based on demographics and habits (No other scores
↳allowed!)
features_to_drop = ["MathScore", "ReadingScore", "WritingScore", "TotalScore",
↳"AvgScore", "Performance"]
X = df.drop(columns=["MathScore", "ReadingScore", "WritingScore", "TotalScore",
↳"AvgScore", "Performance", "ZScore"], errors="ignore")
y = df["MathScore"]

# Encode categorical variables
X_encoded = pd.get_dummies(X, drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2,
↳random_state=42)
# Model training
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction & Evaluation
y_pred = model.predict(X_test)
print(f"R2 Score: {r2_score(y_test, y_pred):.3f}")
print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_pred):.2f} points")

# Feature importance mapped cleanly
importance = pd.DataFrame({'Feature': X_encoded.columns, 'Importance': model.
↳coef_})
print("\nTop 5 Drivers of Math Scores:")
print(importance.sort_values(by='Importance', key=abs, ascending=False).head(5))
```

R2 Score: 0.289

Mean Absolute Error: 10.38 points

Top 5 Drivers of Math Scores:

	Feature	Importance
5	EthnicGroup_group E	12.532591
11	LunchType_standard	12.022905

```

10 ParentEduc_some high school    -5.998080
1          Gender_male            5.126620
4          EthnicGroup_group D    4.732609

```

8 feature importance

```
[33]: importance = pd.Series(model.coef_, index=X_encoded.columns)
```

```

print("--- Top 10 Positive Drivers ---")
print(importance.sort_values(ascending=False).head(10))

print("\n--- Top 10 Negative Drivers ---")
print(importance.sort_values(ascending=True).head(10))

```

```

--- Top 10 Positive Drivers ---
EthnicGroup_group E          12.532591
LunchType_standard          12.022905
Gender_male                  5.126620
EthnicGroup_group D          4.732609
ParentEduc_master's degree   4.388060
PracticeSport_regularly      3.747887
PracticeSport_sometimes      2.115744
ParentEduc_bachelor's degree 2.047897
EthnicGroup_group C          1.956503
WklyStudyHours_> 10          1.743483
dtype: float64

```

```

--- Top 10 Negative Drivers ---
ParentEduc_some high school  -5.998080
TestPrep_none                -4.430319
ParentEduc_high school       -3.723490
WklyStudyHours_< 5           -2.350004
ParentEduc_some college      -1.922212
ParentMaritalStatus_single   -0.439861
ParentMaritalStatus_married  -0.166177
TransportMeans_school_bus     -0.013675
NrSiblings                   0.038868
IsFirstChild_yes             0.368320
dtype: float64

```

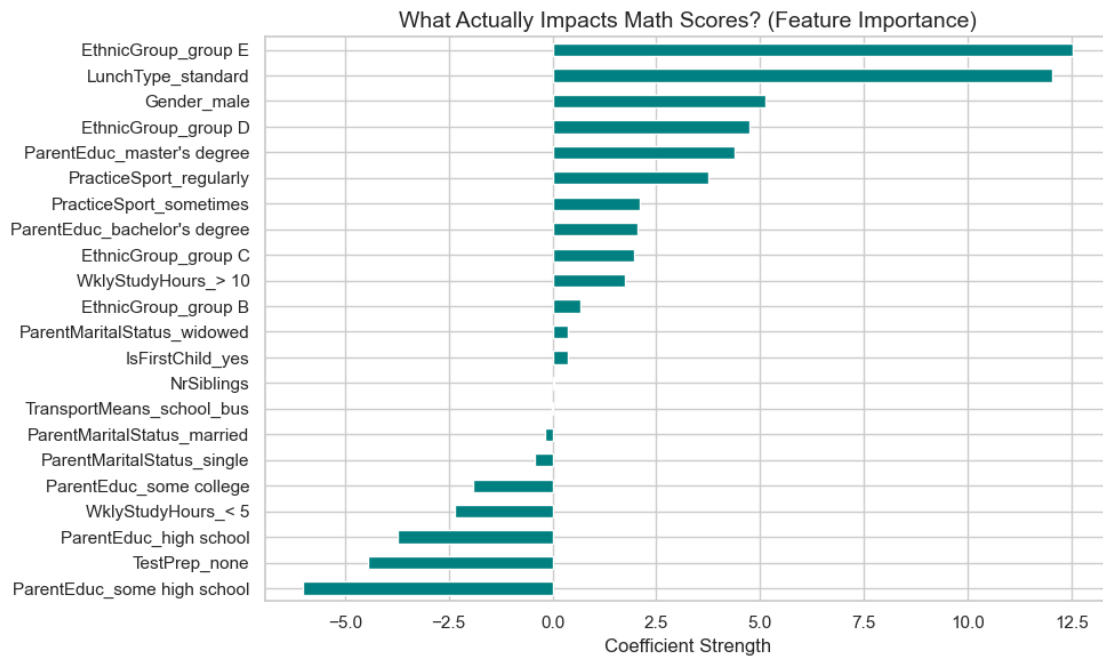
```
[34]: importance = pd.Series(model.coef_, index=X_encoded.columns).sort_values()
```

```

plt.figure(figsize=(10, 6))
importance.plot(kind='barh', color='teal')
plt.title("What Actually Impacts Math Scores? (Feature Importance)")
plt.xlabel("Coefficient Strength")
plt.tight_layout()

```

```
plt.show()
```



8.0.1 Project Conclusions & Recommendations

1. **Intervention Priority:** Math scores show the highest variance and lowest median. Targeted math tutoring is the highest-value intervention.
2. **Test Prep ROI:** Completion of test prep resulted in a statistically significant score increase (~10% improvement). This program should be mandatory for low-performing groups.
3. **Environmental Factors:** Parental education remains a top-3 predictor of success. Schools should provide additional literacy resources to students whose parents do not have higher education.
4. **Model Performance:** Our model explains the variance in Math scores based purely on demographics and habits, showing that student background significantly influences academic baseline.

```
[ ]:
```